

PSE-LPCM-IMPLEMENTATION-01

Fragmented 51% Condensation Layer

Implementation-Ready Specification v0.1

Sebastian Klemm

Engineering specification for post-symbolic computation systems

11. May 2026

Abstract

This specification translates Local-Percolative Collapse Mechanics (LPCM-01) into an implementable, deterministic PSE layer. The layer partitions a bounded phase-space window into local fragments, constructs local topology patches, generates candidate directions, computes normalized support-mass vectors, applies a hysteretic 51% majority gate, links local condensations by seam compatibility, coarse-grains accepted paths, and emits replayable collapse reports. The layer is additive, ablatable, fail-closed, and non-committing. It does not create SemanticCrystals, does not replace TPT/MTL, does not mutate persistent tensors, and does not treat local 51% events as truth. Its purpose is to provide a high-power local-collapse mechanism for fragmented, underdetermined, or topologically incomplete solution landscapes.

Contents

1	Status and Normative Language	4
2	Purpose	4
3	Non-Goals	4
4	Placement in the PSE Stack	4
4.1	Preferred module placement	4
4.2	Layer contract	5
5	Module Layout	5
6	Determinism Requirements	6
6.1	Canonical ordering	6
6.2	Canonical serialization	6
6.3	Numeric representation	6
6.4	Tie policy	6
7	Core Data Types	6
7.1	Primitives	6
7.2	Run descriptor	7
7.3	Fragmented window	7
7.4	Local topology patch	8
7.5	Candidate directions	8
7.6	Support mass	9
7.7	Local condensation	9
7.8	Seam graph and percolation	10
7.9	Coarse-grained output	10
8	Failure and Outcome Model	11
9	Pipeline Semantics	12
9.1	Canonical pipeline	12
9.2	Reference pseudocode	12
10	Support-Mass Computation	13
10.1	Raw support	13
10.2	Normalization	13
10.3	Local majority	13
10.4	Dominance margin	14
11	Partition Policies	14
12	Candidate Generation Policies	14
13	Seam and Percolation	14
13.1	Seam scoring	14
13.2	Percolation search	15
13.3	False percolation prevention	15

14 Coarse-Graining	15
15 Replay and Verification	15
15.1 Replay function	15
15.2 Replay report	15
16 Metrics	16
17 CLI	16
18 Evaluation and Ablation	16
19 Integration with Validation Runner	17
20 NCTCS and Metatron Interaction	17
21 Feature Flags	17
22 Implementation Phases	18
22.1 Phase A: skeleton and types	18
22.2 Phase B: deterministic partitioning	18
22.3 Phase C: topology patches	18
22.4 Phase D: candidates and support mass	18
22.5 Phase E: local gate	18
22.6 Phase F: seam graph and percolation	18
22.7 Phase G: coarse-grain and report	18
22.8 Phase H: CLI, eval, validation integration	18
23 Tests	19
23.1 Unit tests	19
23.2 Integration tests	19
23.3 Negative tests	19
24 Public Exports	19
25 Definition of Done	20
26 Coding Agent Prompt	20
27 Final Invariant	21

1 Status and Normative Language

This document is a normative implementation specification. The terms **MUST**, **MUST NOT**, **SHOULD** and **MAY** are binding for conformance.

The canonical implementation name is:

PSE-LPCM-IMPLEMENTATION-01

The canonical feature name is:

lpcm-fragment-collapse

2 Purpose

The implementation provides a local-percolative collapse layer over existing post-symbolic traversal and topology artifacts. It converts fragmented local uncertainty into traceable local condensation candidates, and then into higher-order collapse candidates only if seam, boundary, replay, and validation gates permit.

Primary invariant

A 51% local condensation bit is not a truth value, not a commit authority, and not a final output. It is a local dominance signal over normalized admissible support mass inside a deterministic fragment.

3 Non-Goals

The implementation **MUST NOT** do any of the following:

- replace PSE-Core, TPT, MTL, Projection, NCTCS, Metatron, or the Validation Runner;
- create `SemanticCrystal`, `FinalizedEmission`, or persistent tensor updates;
- bypass existing topology, replay, trace, or gate layers;
- interpret local majority as global truth;
- use platform-dependent floating point arithmetic in audit paths;
- introduce network access, nondeterministic IO, wall-clock time, or random seeds not recorded in the run descriptor.

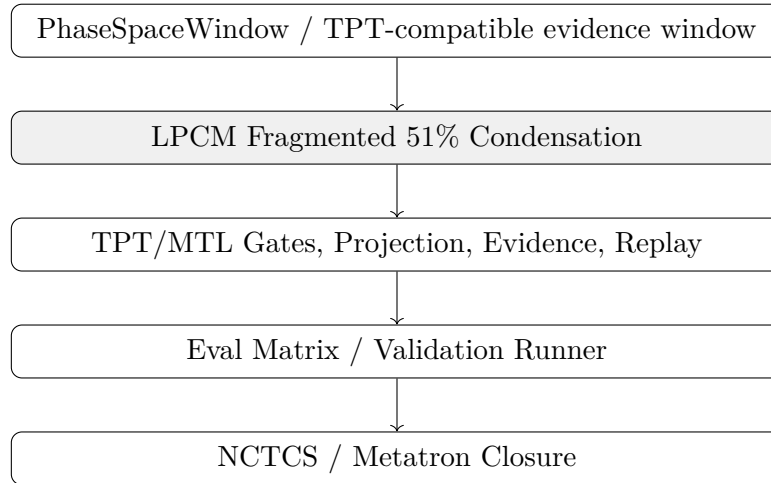
4 Placement in the PSE Stack

4.1 Preferred module placement

The preferred implementation location is:

```
crates/pse-traverse/src/lpcm/
```

The layer is downstream from TPT/MTL-compatible phase-space windows and upstream from projection, evaluation, validation, NCTCS, and Metatron closure.



4.2 Layer contract

LPCM **MUST** read existing windows and evidence. It **MAY** construct local topology patches and candidate directions. It **MUST** emit only reports, diagnostics, and optional candidate bundles. It **MUST NOT** mutate the source window.

5 Module Layout

```

crates/pse-traverse/src/lpcm/
  mod.rs
  primitives.rs
  run_descriptor.rs
  evidence.rs
  fragment.rs
  partition.rs
  topology_patch.rs
  candidate.rs
  support_mass.rs
  local_gate.rs
  seam.rs
  percolation.rs
  coarse_grain.rs
  report.rs
  replay.rs
  metrics.rs
  adapters.rs
  pipeline.rs
  tests.rs

tools/pse-traverse-lpcm-cli/
  Cargo.toml
  src/main.rs

examples/lpcm_minimal_window.json
examples/lpcm_fragmented_gap.json
  
```

If repository conventions require another crate, the same module boundaries **MUST** be preserved.

6 Determinism Requirements

6.1 Canonical ordering

All keyed collections in audit paths **MUST** use `BTreeMap` or `BTreeSet`. All vectors that participate in hashing **MUST** be sorted by stable identifiers before serialization.

6.2 Canonical serialization

Every public artifact **MUST** be serialized canonically before hashing. The canonical serialization **SHOULD** be JCS-compatible if the repository already uses JCS. Hashes **MUST** use SHA-256.

6.3 Numeric representation

Audit-path scores **MUST** use existing fixed/canonical numeric types. If no shared type is available in the target crate, implement a rational fixed-point wrapper with normalized numerator/denominator and deterministic overflow checks. Platform `f32/f64` **MUST NOT** be used in report-critical computations.

6.4 Tie policy

If two candidate directions have identical maximal support mass, the fragment **MUST** not pass the local majority gate unless a configured deterministic tie-break proof exists. Default behavior: `TieNoCondense`.

7 Core Data Types

7.1 Primitives

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = CanonicalNumber;

pub struct EvidenceRef {
    pub kind: String,
    pub address: Hash256,
    pub relation: String,
}

pub struct ContentAddressed<T> {
    pub id: Hash256,
    pub value: T,
}
```

7.2 Run descriptor

```
pub struct LpcmRunDescriptor {
  pub run_id: StableId,
  pub lpcm_version: String,           // "PSE-LPCM-IMPLEMENTATION-01"
  pub source_window_hash: Hash256,
  pub source_topology_hash: Option<Hash256>,
  pub seed: u64,
  pub thresholds: LpcmThresholds,
  pub policies: LpcmPolicies,
  pub operator_versions: BTreeMap<String, String>,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct LpcmThresholds {
  pub theta_activate: Fixed,          // default 0.51
  pub theta_deactivate: Fixed,        // default 0.50 or configured below
  theta_activate
  pub min_dominance_margin: Fixed,     // optional safety margin
  pub seam_min: Fixed,
  pub percolation_min_path_strength: Fixed,
  pub coarse_grain_min_support: Fixed,
  pub max_false_local_collapse_rate: Fixed,
}

pub struct LpcmPolicies {
  pub partition_policy: PartitionPolicy,
  pub topology_policy: TopologyPolicy,
  pub candidate_policy: CandidatePolicy,
  pub support_policy: SupportPolicy,
  pub tie_policy: TiePolicy,
  pub percolation_policy: PercolationPolicy,
  pub coarse_grain_policy: CoarseGrainPolicy,
  pub fail_on_missing_provenance: bool,
  pub require_replay_identity: bool,
}
```

7.3 Fragmented window

```
pub struct FragmentedPhaseSpaceWindow {
  pub window_id: Hash256,
  pub source_window_hash: Hash256,
  pub fragments: Vec<FragmentRef>,
  pub adjacency: BTreeMap<Hash256, Vec<Hash256>>,
  pub partition_report_ref: Hash256,
}

pub struct FragmentRef {
  pub fragment_id: Hash256,
  pub ordinal: u64,
  pub evidence_refs: Vec<EvidenceRef>,
  pub boundary_hash: Hash256,
```

```
pub carrier_hash: Option<Hash256>,
}
```

7.4 Local topology patch

```
pub struct LocalTopologyPatch {
  pub patch_id: Hash256,
  pub fragment_id: Hash256,
  pub vertices: Vec<TopologyVertex>,
  pub edges: Vec<TopologyEdge>,
  pub simplices: Vec<TopologySimplex>,
  pub boundary_model_hash: Hash256,
  pub invariant_summary: TopologyInvariantSummary,
  pub topology_guard: GateResult,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct TopologyVertex {
  pub vertex_id: Hash256,
  pub coordinate_hash: Hash256,
  pub feature_hash: Hash256,
  pub evidence_ref: EvidenceRef,
}

pub struct TopologyEdge {
  pub edge_id: Hash256,
  pub a: Hash256,
  pub b: Hash256,
  pub weight: Fixed,
}

pub struct TopologySimplex {
  pub simplex_id: Hash256,
  pub vertices: Vec<Hash256>,
  pub orientation_hash: Hash256,
}
```

7.5 Candidate directions

```
pub struct CandidateDirection {
  pub candidate_id: Hash256,
  pub fragment_id: Hash256,
  pub direction_kind: CandidateDirectionKind,
  pub direction_signature_hash: Hash256,
  pub proposed_boundary_delta_hash: Hash256,
  pub proposed_seam_delta_hash: Hash256,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum CandidateDirectionKind {
  CompleteGap,
```

```

    ContinuePath,
    ContractRegion,
    RefineMesh,
    BridgeBoundary,
    RemoveContradiction,
    HoldDiagnostic,
}

```

7.6 Support mass

```

pub struct SupportFactorVector {
    pub candidate_id: Hash256,
    pub adjacency: Fixed,
    pub topology: Fixed,
    pub seam: Fixed,
    pub resonance: Fixed,
    pub provenance: Fixed,
    pub gate: Fixed,
    pub hysteresis: Fixed,
}

pub struct SupportMassVector {
    pub vector_id: Hash256,
    pub fragment_id: Hash256,
    pub weights: BTreeMap<Hash256, Fixed>, // candidate_id -> normalized mass
    pub raw_factors: Vec<SupportFactorVector>,
    pub denominator: Fixed,
    pub valid: bool,
    pub failure: Option<LpcmFailureKind>,
}

```

7.7 Local condensation

```

pub struct LocalCondensationBit {
    pub bit_id: Hash256,
    pub fragment_id: Hash256,
    pub winning_candidate_id: Option<Hash256>,
    pub z_max: Fixed,
    pub runner_up: Fixed,
    pub dominance_margin: Fixed,
    pub active: bool,
    pub status: LocalCondensationStatus,
    pub failure: Option<LpcmFailureKind>,
}

pub enum LocalCondensationStatus {
    Dormant,
    CondensedLocal,
    TieNoCondense,
    ZeroSupport,
    TopologyRejected,
}

```

```

    ProvenanceRejected,
}

pub struct LocalCondensationCandidate {
    pub local_candidate_id: Hash256,
    pub fragment_id: Hash256,
    pub patch_id: Hash256,
    pub candidate_direction_id: Hash256,
    pub support_vector_id: Hash256,
    pub bit_id: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

7.8 Seam graph and percolation

```

pub struct SeamEdge {
    pub edge_id: Hash256,
    pub from_local_candidate: Hash256,
    pub to_local_candidate: Hash256,
    pub seam_score: Fixed,
    pub boundary_score: Fixed,
    pub carrier_score: Fixed,
    pub replay_compatible: bool,
    pub accepted: bool,
    pub failure: Option<LpcmFailureKind>,
}

pub struct SeamPercolationGraph {
    pub graph_id: Hash256,
    pub nodes: Vec<Hash256>,
    pub edges: Vec<SeamEdge>,
    pub connected_components: Vec<PercolationComponent>,
}

pub struct PercolativePath {
    pub path_id: Hash256,
    pub node_sequence: Vec<Hash256>,
    pub edge_sequence: Vec<Hash256>,
    pub path_strength: Fixed,
    pub crosses_completion_boundary: bool,
    pub replay_ready: bool,
    pub accepted: bool,
}

```

7.9 Coarse-grained output

```

pub struct CoarseGrainCondensate {
    pub condensate_id: Hash256,
    pub level: u32,
    pub source_path_refs: Vec<Hash256>,
    pub aggregate_support: BTreeMap<Hash256, Fixed>,
}

```

```

    pub z_aggregate: Fixed,
    pub active: bool,
    pub stability_score: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct HierarchicalCollapseReport {
    pub report_id: Hash256,
    pub run_descriptor_hash: Hash256,
    pub source_window_hash: Hash256,
    pub fragments_hash: Hash256,
    pub local_patch_refs: Vec<Hash256>,
    pub support_vector_refs: Vec<Hash256>,
    pub local_candidate_refs: Vec<Hash256>,
    pub seam_graph_ref: Hash256,
    pub percolative_path_refs: Vec<Hash256>,
    pub coarse_grain_refs: Vec<Hash256>,
    pub metrics: LpcmMetrics,
    pub outcome: LpcmOutcome,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

8 Failure and Outcome Model

```

pub enum LpcmFailureKind {
    NoProvenance,
    NonTriangulablePatch,
    TopologyGuardFailed,
    NoCandidateDirections,
    ZeroAdmissibleSupport,
    NoMajority,
    TieNoCondense,
    OscillatoryFlip,
    SeamBreak,
    FalsePercolation,
    CoarseGrainConflict,
    ReplayMismatch,
    InvalidDescriptor,
}

pub enum LpcmOutcome {
    NoFragments,
    DiagnosticHold,
    LocalOnly,
    SeamLinked,
    Percolating,
    CoarseGrained,
    CollapseCandidateReady,
    InvalidInput,
    DeterminismViolation,
}

```

9 Pipeline Semantics

9.1 Canonical pipeline

1. Load and canonicalize a bounded input window.
2. Validate run descriptor, policies, thresholds, operator versions, and evidence references.
3. Partition the source window into deterministic fragments.
4. Construct local topology patches for every fragment.
5. Run topology guards; reject non-triangulable or non-provenanced patches.
6. Generate candidate directions per accepted patch.
7. Compute raw support factors and normalized support-mass vectors.
8. Apply hysteretic 51% local majority gate.
9. Emit local condensation candidates for activated fragments.
10. Build seam compatibility graph across local candidates.
11. Search accepted percolative paths.
12. Coarse-grain compatible paths into higher-order condensates.
13. Emit HierarchicalCollapseReport.
14. Verify replay identity.

9.2 Reference pseudocode

```
pub fn run_lpcm(
    rd: LpcmRunDescriptor,
    input: LpcmInputWindow,
) -> LpcmResult<HierarchicalCollapseReport> {
    validate_descriptor(&rd)?;
    let canonical_input = canonicalize_input(input)?;

    let fragmented = partition_window(&rd, &canonical_input)?;
    let mut patch_reports = Vec::new();
    let mut local_candidates = Vec::new();

    for fragment in sorted_fragments(&fragmented) {
        let patch = build_local_topology_patch(&rd, &fragment)?;
        if !patch.topology_guard.passed {
            patch_reports.push(hold_patch(patch, LpcmFailureKind::
TopologyGuardFailed));
            continue;
        }

        let candidates = generate_candidate_directions(&rd, &patch)?;
        let support = compute_support_mass(&rd, &patch, &candidates)?;
        let bit = evaluate_local_condensation(&rd, &support)?;

        patch_reports.push(record_patch(patch, support, bit.clone()));
    }
}
```

```

        if bit.active {
            local_candidates.push(build_local_candidate(&rd, &bit, &candidates)?)
        }
    }

    let seam_graph = build_seam_percolation_graph(&rd, &local_candidates)?;
    let paths = find_percolative_paths(&rd, &seam_graph)?;
    let condensates = coarse_grain_paths(&rd, &paths)?;

    let report = build_hierarchical_collapse_report(
        &rd,
        &fragmented,
        patch_reports,
        local_candidates,
        seam_graph,
        paths,
        condensates,
    )?;

    verify_lpcm_replay(&rd, &canonical_input, &report)?;
    Ok(report)
}

```

10 Support-Mass Computation

10.1 Raw support

For candidate c_i in fragment U , compute a raw support value

$$\tilde{w}_i^U = F_A(i) \cdot F_T(i) \cdot F_S(i) \cdot F_R(i) \cdot F_P(i) \cdot F_G(i) \cdot F_H(i).$$

Every factor **MUST** be in $[0, 1]$ and represented as **Fixed**. A zero factor vetoes that support channel.

10.2 Normalization

If

$$D_U = \sum_i \tilde{w}_i^U > 0,$$

then

$$w_i^U = \frac{\tilde{w}_i^U}{D_U}.$$

If $D_U = 0$, the fragment **MUST** emit **ZeroAdmissibleSupport**.

10.3 Local majority

Let

$$Z_U = \max_i w_i^U.$$

A fragment passes the local gate if and only if:

$$Z_U \geq \theta_{activate}$$

and no tie or explicit veto exists.

10.4 Dominance margin

Let w_1^* be the winning mass and w_2^* the second-largest mass. The dominance margin is

$$\Delta_U = w_1^* - w_2^*.$$

If `min_dominance_margin` is non-zero, the gate also requires

$$\Delta_U \geq \Delta_{min}.$$

11 Partition Policies

A conformant implementation **MUST** implement at least one deterministic partition policy and **SHOULD** implement all policies below if the necessary input data is available.

```
pub enum PartitionPolicy {
    FixedGrid { cells_per_axis: u32 },
    GraphKHop { k: u32 },
    CarrierWindow { width: u32, stride: u32 },
    TopologyAdaptive { max_vertices: u32, max_overlap: u32 },
}
```

Partitioning **MUST** be reproducible from input hash, descriptor, and operator versions. Overlap between fragments **MUST** be explicit.

12 Candidate Generation Policies

```
pub enum CandidatePolicy {
    BoundaryCompletion,
    MeshRefinement,
    SeamBridge,
    ContradictionRemoval,
    HybridDeclared(Vec<CandidateDirectionKind>),
}
```

Candidate generation **MUST** be local to the patch and **MUST** produce stable candidate identifiers. The implementation **MUST** cap candidate count per patch using descriptor-bound limits to prevent unbounded explosion.

13 Seam and Percolation

13.1 Seam scoring

The seam score between two local candidates is computed from declared factors:

$$\Gamma(K_U, K_V) = \alpha_b B + \alpha_c C + \alpha_t T + \alpha_r R + \alpha_p P$$

where weights sum to one. All factors and weights **MUST** be descriptor-bound.

13.2 Percolation search

The default search policy **MUST** be deterministic breadth-first search over sorted node identifiers, followed by stable scoring of accepted paths. Path scoring **MUST** account for:

- node support;
- edge seam score;
- completion-boundary crossing;
- replay readiness;
- contradiction absence.

13.3 False percolation prevention

A connected graph path **MUST NOT** count as percolating if it fails replay, seam, boundary, or topology compatibility. Such a path **MUST** be reported as **FalsePercolation**.

14 Coarse-Graining

Coarse-graining aggregates accepted percolative paths into higher-order condensates. It **MUST** preserve provenance, support mass, and replay identity. It **MUST** produce a new support vector at the next scale.

```
pub enum CoarseGrainPolicy {
    WeightedSupportMean,
    MinEdgeBottleneck,
    MajorityMonotone,
}
```

The default policy **MUST** be **MajorityMonotone**: increasing support for a candidate at the lower scale must not reduce its aggregate support unless an explicit negative gate fires.

15 Replay and Verification

15.1 Replay function

```
pub fn verify_lpcm_replay(
    rd: &LpcmRunDescriptor,
    input: &LpcmInputWindow,
    expected: &HierarchicalCollapseReport,
) -> LpcmReplayReport;
```

Replay **MUST** re-run partitioning, topology patching, candidate generation, support-mass computation, local gating, seam graph construction, percolation search, and coarse-graining. It **MUST** compare canonical report bytes or canonical content hashes.

15.2 Replay report

```
pub struct LpcmReplayReport {
    pub replay_id: Hash256,
```

```

pub expected_report_id: Hash256,
pub actual_report_id: Hash256,
pub byte_identical: bool,
pub hash_identical: bool,
pub mismatch_kind: Option<LpcmFailureKind>,
}

```

16 Metrics

```

pub struct LpcmMetrics {
pub fragment_count: u64,
pub triangulable_patch_rate: Fixed,
pub candidate_direction_count_mean: Fixed,
pub local_condensation_rate: Fixed,
pub mean_dominance_margin: Fixed,
pub seam_consistency_rate: Fixed,
pub percolation_path_count: u64,
pub strongest_path_score: Fixed,
pub coarse_grain_stability: Fixed,
pub false_local_collapse_rate: Fixed,
pub replay_identity: bool,
}

```

Metrics **SHOULD** be registered with the evaluation matrix under a dedicated preset:

lpcm-fragment-collapse

17 CLI

A CLI **SHOULD** be provided for direct debugging and benchmark integration.

```

pse-traverse-lpcm inspect <window.json>
pse-traverse-lpcm partition <window.json> --out fragments.json
pse-traverse-lpcm collapse <window.json> --rd lpcm_rd.json --out collapse_report.
    json
pse-traverse-lpcm replay --rd lpcm_rd.json --input window.json --report
    collapse_report.json
pse-traverse-lpcm verify collapse_report.json
pse-traverse-lpcm ablate <window.json> --without-lpcm --with-lpcm --out ablation.
    json

```

The CLI **MUST** exit non-zero on invalid input, replay mismatch, or descriptor violation.

18 Evaluation and Ablation

The implementation **MUST** support ablation. A validation profile **SHOULD** compare:

- baseline topology processing without LPCM;
- LPCM with local 51% gate disabled;

- LPCM with local gate enabled but seam percolation disabled;
- full LPCM with seam percolation and coarse-graining;
- full LPCM with replay verification.

Required reported deltas:

- candidate reduction ratio;
- accepted path strength delta;
- false local collapse rate;
- runtime overhead;
- replay identity;
- downstream gate pass delta.

19 Integration with Validation Runner

The Validation Runner **SHOULD** be able to consume an LPCM report if present. It **MUST NOT** require LPCM for generic validation profiles unless the profile explicitly declares it.

```
pse-validation-runner run --profile domain \
  --domain-manifest validation_domains/<domain>/manifest.json \
  --with-lpcm \
  --out validation_runs/<run_id>
```

If `-with-lpcm` is set, the final report **SHOULD** include:

- `lpcm_report_id`;
- `lpcm_outcome`;
- `local_condensation_rate`;
- `false_local_collapse_rate`;
- `strongest_path_score`;
- `replay_identity`;
- `ablation_delta_vs_no_lpcm` if available.

20 NCTCS and Metatron Interaction

LPCM outputs **MAY** be referenced by NCTCS and Metatron closure reports as diagnostic or pre-closure evidence. LPCM **MUST NOT** directly produce a `MacroControlState` or `HolisticEigenmodeState`. Those remain closure-layer responsibilities.

21 Feature Flags

```
[features]
lpcm-fragment-collapse = []
lpcm-cli = ["lpcm-fragment-collapse"]
lpcm-eval = ["lpcm-fragment-collapse"]
lpcm-validation-runner = ["lpcm-fragment-collapse"]
```

The crate **MUST** compile with LPCM disabled. Public types needed for report reading **MAY** remain available behind a lightweight report feature if repository conventions require it.

22 Implementation Phases

22.1 Phase A: skeleton and types

Implement module layout, public exports, descriptor types, threshold/policy types, failure/outcome enums, and canonical hashing helpers.

22.2 Phase B: deterministic partitioning

Implement at least `FixedGrid` and `GraphKHop` partition policies or the closest compatible policies for existing PSE window types.

22.3 Phase C: topology patches

Implement local topology patch construction and topology guards. Existing TPT/MTL types **SHOULD** be reused through adapters.

22.4 Phase D: candidates and support mass

Implement candidate generation, factor computation, normalization, zero-support handling, tie handling, and dominance-margin computation.

22.5 Phase E: local gate

Implement hysteretic 51% local gate and local condensation candidate emission.

22.6 Phase F: seam graph and percolation

Implement seam scoring, deterministic graph construction, connected components, path search, path scoring, and false-percolation handling.

22.7 Phase G: coarse-grain and report

Implement coarse-graining, metric aggregation, `HierarchicalCollapseReport`, content addressing, replay, verify.

22.8 Phase H: CLI, eval, validation integration

Implement CLI commands, eval preset registration, validation-runner optional integration, README, CHANGELOG, examples.

23 Tests

23.1 Unit tests

1. support_mass_normalizes_to_one
2. zero_support_emits_failure
3. local_51_gate_passes_above_threshold
4. local_51_gate_rejects_below_threshold
5. tie_does_not_condense_by_default
6. dominance_margin_required_when_configured
7. missing_provenance_rejected
8. topology_guard_failure_blocks_candidates
9. seam_edge_requires_replay_compatibility
10. false_percolation_is_not_accepted
11. coarse_grain_preserves_majority_monotonicity
12. content_addresses_change_when_content_changes

23.2 Integration tests

1. minimal_window_produces_diagnostic_hold
2. fragmented_gap_produces_local_condensations
3. seam_compatible_candidates_percolate
4. seam_incompatible_candidates_do_not_percolate
5. collapse_report_replay_byte_identical
6. cli_collapse_then_replay_ok
7. eval_matrix_registers_lpcm_metrics
8. validation_runner_with_lpcm_writes_report_ref

23.3 Negative tests

1. local majority cannot create `SemanticCrystal`;
2. local majority cannot create `MacroControlState`;
3. local majority cannot create `HolisticEigenmodeState`;
4. non-canonical vector ordering changes are rejected by replay;
5. platform float usage in audit path is rejected by static check or review test;
6. no-provenance input cannot contribute to support mass.

24 Public Exports

```
pub mod lpcm;

pub use lpcm::{
    LpcmRunDescriptor,
    LpcmThresholds,
    LpcmPolicies,
    FragmentedPhaseSpaceWindow,
```

```

    LocalTopologyPatch,
    CandidateDirection,
    SupportMassVector,
    LocalCondensationBit,
    LocalCondensationCandidate,
    SeamPercolationGraph,
    PercolativePath,
    CoarseGrainCondensate,
    HierarchicalCollapseReport,
    LpcmMetrics,
    LpcmOutcome,
    verify_lpcm_replay,
    run_lpcm,
};

```

25 Definition of Done

An implementation is complete when:

1. `cargo fmt -all - -check` passes;
2. `cargo clippy -workspace -all-targets -locked` passes with no warnings;
3. `cargo test -workspace -locked` passes;
4. `cargo doc -workspace -no-deps -locked` passes without warnings;
5. LPCM compiles both enabled and disabled;
6. examples produce deterministic reports;
7. CLI collapse, replay, and verify work end-to-end;
8. replay is byte-identical for the same input and descriptor;
9. local 51% events never create commit-layer artifacts;
10. eval metrics and optional validation-runner integration are documented;
11. README and CHANGELOG contain a shipped or experimental status entry.

26 Coding Agent Prompt

PATCH-ID: PSE-LPCM-IMPLEMENTATION-01

TITLE: Fragmented 51% Condensation Layer

Implement the LPCM layer as an additive, ablatable, fail-closed layer over existing PSE traversal/topology artifacts.

PRIMARY SPEC:

- `pse_lpcm_implementation_spec_v0_1.pdf`

FOUNDATION:

- `local_percolative_collapse_mechanics_v0_1.pdf`

GOAL:

Partition a bounded phase-space/evidence window into deterministic fragments, build local topology patches, generate candidate directions, compute

normalized support-mass vectors, apply a hysteretic 51% local majority gate, link accepted local condensations through seam percolation, coarse-grain compatible paths, and emit a replayable HierarchicalCollapseReport.

MUST:

- add crates/pse-traverse/src/lpcm/ modules or equivalent repository-native placement;
- implement all public core types from the spec;
- use deterministic ordering, canonical serialization, content addressing, and fixed/canonical numeric types;
- implement support-mass normalization and 51% gate;
- implement seam graph, percolation paths, coarse-graining, metrics, replay, verify;
- add CLI pse-traverse-lpcm with inspect, partition, collapse, replay, verify, ablate;
- add examples and tests;
- register eval metrics if the eval matrix crate is available;
- integrate with validation runner only as optional `--with-lpcm` path.

MUST NOT:

- no SemanticCrystal creation;
- no FinalizedEmission creation;
- no MacroControlState or HolisticEigenmodeState creation;
- no mutation of source windows or persistent tensors;
- no interpretation of local 51% as truth;
- no platform floats in audit paths;
- no nondeterministic ordering, random IO, or network dependency.

SUCCESS:

- cargo fmt, clippy, test, doc all green;
- collapse -> replay -> verify works on examples;
- negative tests prove local majority cannot materialize final artifacts;
- same input + same descriptor yields byte-identical report.

27 Final Invariant

$$\text{Local51}(U) \Rightarrow \text{Candidate}(U)$$

$$\text{Candidate}(U) \not\Rightarrow \text{Truth}$$

$$\text{SeamPercolation} + \text{Replay} + \text{Gates} \Rightarrow \text{CollapseCandidateReady}$$

End condition

LPCM gives PSE a local condensation drill. It may produce bounded local breakthroughs and percolative paths through fragmented solution spaces, but every such path remains a candidate until existing validation and closure layers accept it.
